Interview-1

Agenda:

1. Continuous sum Query from i to last index
2. Continuous sum query from i to j
3. Rain Water Trap
4. Search in Row-wise, Column-wise Sorted Matrix

Q1: Given an integer array A where every element is 0, return the final array after performing multiple queries as follows.

Query (i, x): Add x to all the numbers from index i to N-1

Index	0	1	2	3	4	5	6
Array	0	0	0	0	0	0	0

Query(1,3)	Q1:	:	+3	+3	+3	+3	+3	+3
Query(4,-2)	Q2:	:	:	:	:	-2	-2	-2
Query(3,1)	Q3:	:	:	:	+1	+1	+1	+1
	Ans[] =	0	3	3	4	2	2	2

Sol. Efficient Trick → Difference Array

instead of adding x repeatedly (O(N·Q)), we do, diff[i] += x.

final array is obtain using prefix-sum:

A[i] = A[i] + diff[i] + diff[i-1] + dif[i-2] - - -

Code:

```
public int[] solve (int N, int [][] queries) {
int [] diff = new int [N];          //(diff array initialised with 0)
for (int q=0; q < queries.length; q++){
        int i = queries[q][0];      // apply updates at that index i
        int x = queries[q][1];
        diff[i] += x;
}
int [] A = new int[N]               // Build final array using prefix sum
A[0] = diff[0]
for (int i=1; i<N; i++){            (Prefix sum spread the applied sum from that index to the end
        A[i] = A[i-1] + diff[i];
}
return A;
}
```

Q.2

[i, j, x]

i → start index

j → last index

0	0	0	0	0	0	0
	+3	+3	+3			
-1	-1	-1	-1	-1		
		+4				
		+3	+3	+3	+3	

Sol.

```
int [] diff = new int[N]
for (int q = 0 ; q < Queries.length ; q++){
        int i = Queries [q][0];
        int j = Queries [q][1];
        int x = Queries[q][2];

        diff [i] += x                    // Because prefix sum
        if (j+1 < N) diff [j+1] -= x      has spreading effect

        int[] pref = new int[N];
        pref[0] = diff[0];
            for (int i=0; i<N ; i++) {
                pref [i] = pref[i-1] + diff[i];
            }
            return pref;
        }
```

Refer Beggar problem Interview-1

Q.3 # Rain water trapped Problem

```
Public int trap (final int[] A){
    int n = A.length;
    int[] LM = new int [n+1];
    int[] RM = new int [n+1];
    for (int i=0 ; i<n ; i++) {
            LM[i+1] = Math.max (LM[i], A[i]);
    }
    for (int i= n-1 ; i>=0 ; i--){
            RM[i]= Math.max (RM[i+1], A[i]);
    int totalWater =0;
    For (int i=0 ; i<n ; i++){
            int waterLevel = Math.min (LM[i], RM[i]);
            int trapped = waterLevel - A[i];
            if (trapped >0) { totalWater += trapped;
    }
    return totalWater.
```